PROJECT REPORT

LAB211

Project Title: Mountain Hiking Challenge Registration
Student: Nguyen Hoang Minh
Student ID: SE203563
Instructor: Ms. Le Vo Minh Thu

Contents

1. Introduction 1
   1.1. Context 1
   1.2. Objectives 1
   1.3. Requirements. 1
2. System Analysis and Design 1
   2.1. Use Case Diagram 1
   2.2. Class Diagram 2
3. Architecture & Implementation 3
4. Technical Highlights 3
5. Results & Screenshots 4
   - Main Menu Interface 4
   - Function 1: New Registration (with Validation) 7
   - Function 2: Update Registration 9
   - Function 3: Display registration list 10
   - Function 4: Delete registration 11
   - Function 5: Search participants by name 12
   - Function 6: Filter data by campus 13
   - Function 7: Statistics of registration number by location 14
   - Function 8: Save to file “registration.dat” 15
   - Function 9: Save Prompt on Exit 16
6. Self-Evaluation Checklist 17

List of Picture

Picture 1. Use case diagram 1
Picture 2. UML diagram 2

List of Table

Table 1. Self - Evaluation 17

1. Introduction

1.1. Context

FPT University, with sponsorship from Viettel and VNPT, has launched the "Because a Generation of Intellectuals Dares to Think and Do" movement. A registration management program for FPT University students is required to support this initiative.

1.2. Objectives

The program will allow students from all campuses nationwide to register for mountain climbing events, selecting from a provided list of mountains. Key features include:

- Allowing new student registrations.
- Managing registration data.
- Collecting and displaying statistical information related to the registrations.

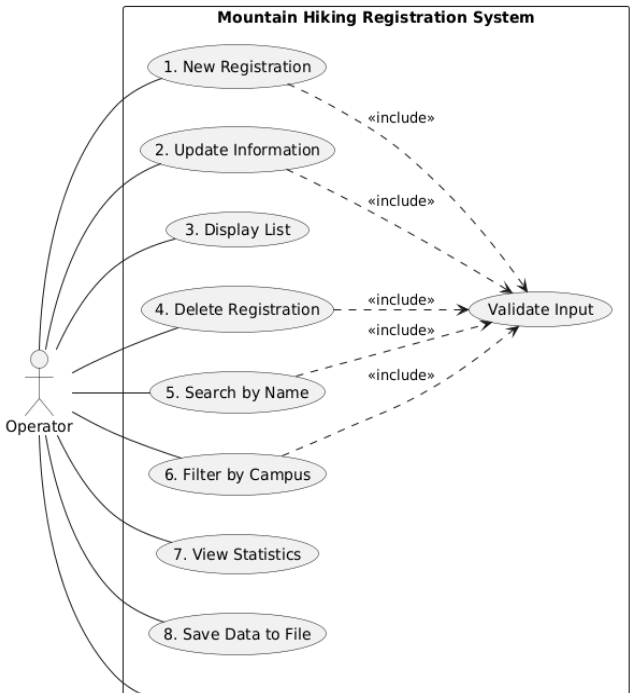
1.3. Requirements.

Design and develop the program using object-oriented programming (OOP) principles, including abstraction, polymorphism, encapsulation, and inheritance.

2. System Analysis and Design

2.1. Use Case Diagram

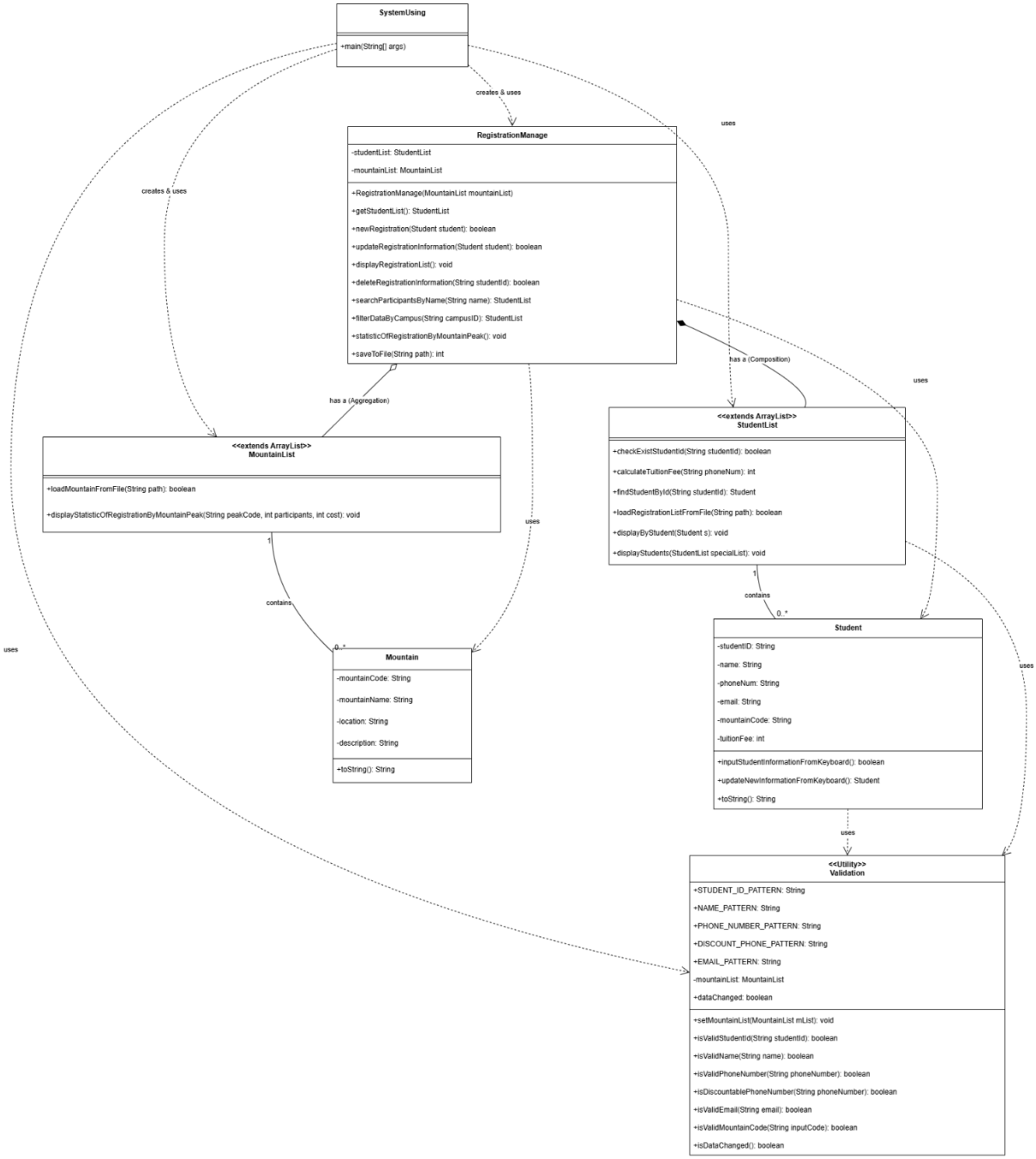
This diagram illustrates the interactions between the primary actor (Operator) and the system's main functionalities. All data input functions require validation, represented by the <<include>> relationship.



Picture 1. Use case diagram

2.2. Class Diagram

The class diagram details the structure of classes and their relationships, strictly adhering to object-oriented design principles.



Picture 2. UML diagram

3. Architecture & Implementation

The project is organized into logically classes structure to ensure clarity and maintainability, adhering to standard Java naming conventions.

- *mountainhiking* (Root Package):
    - *SystemUsing.java*: The main class containing the main method, responsible for controlling the application's primary flow (menu, user choices).
    - *RegistrationManage.java*: The central business logic class, coordinating actions between data lists and objects.
    - *Student.java*, *Mountain.java*: Model classes defining the data structure for core objects.
    - *StudentList.java*, *MountainList.java*: Collection classes that extend ArrayList to manage lists and provide specialized manipulation methods.
    - *Validation.java*: A utility class containing all data validation logic, using REGEX constants for robust input checking.

4. Technical Highlights

The project implements several techniques to optimize code, enhance reusability, and ensure a good user experience.

- **Centralized Validation:** All input validation logic is consolidated within the Validation class. This design allows for easy modification of validation rules (e.g., updating a REGEX) in a single location without affecting other parts of the system.
- **Smart Update Logic:** The "Update" feature (function 2) allows the user to press Enter to skip a field, thereby keeping the old data. This is implemented by reading new input into a temporary variable and only assigning it to the object's attribute if the variable is not empty and is valid.
- **Automatic Fee Calculation:** The participation fee (tuitionFee) is calculated automatically. The base fee is 6,000,000 VND and is discounted by 35% if the student's phone number belongs to the Viettel or VNPT networks, as determined by a REGEX check.
- **Data State Management:** The system uses a boolean flag, Validation.dataChanged, to track whether any data has been modified (add/update/delete). When the user chooses to exit, the program checks this flag and prompts "Do you want to save changes?", preventing accidental data loss.

5. Results & Screenshots

The program has successfully implemented all required functionalities and passed all test cases.

- **Main Menu Interface**

  Code:

```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Main.java to edit this template
 */
package mountainhiking;

import java.util.Scanner;

/**
 *
 * @author asus
 */
public class SystemUsing {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        // TODO code application logic here
        Scanner sc = new Scanner(System.in);
        MountainList mountainData = new MountainList();
        StudentList studentList = new StudentList();
        // Load mountain data from file
        if (mountainData.loadMountainFromFile("MountainList.csv")) {
            System.out.println("Mountain data loaded successfully, " + mountainData.size() + " records found.");
        } else {
            System.out.println("Failed to load mountain data.");
        }
        Validation.setMountainList(mountainData);
        RegistrationManage registrationManage = new RegistrationManage(mountainData);
        if (studentList.loadRegistrationListFromFile("RegistrationList.dat")) {
            System.out.println("Registration data loaded successfully, " + studentList.size() + " records found.");
            registrationManage.getStudentList().addAll(studentList);
        } else {
            System.out.println("No previous registration data found.");
        }
        int choice;
        do {
            System.out.println("\n=====================================================");
            System.out.println("=       MOUNTAIN HIKING CHALLENGE REGISTRATION      =");
            System.out.println("=====================================================");
            System.out.println("| 1. New Registration                               |");
            System.out.println("| 2. Update Registration Information                |");
            System.out.println("| 3. Display Registered List                        |");
```

```java
            System.out.println("| 4. Delete Registration Information                    |");
            System.out.println("| 5. Search Participants by Name                        |");
            System.out.println("| 6. Filter Data by Campus                              |");
            System.out.println("| 7. Statistics of Registration Numbers by Location     |");
            System.out.println("| 8. Save Data to File                                  |");
            System.out.println("| 9. Exit the Program                                   |");
            System.out.println("=========================================================");
            System.out.print("Please choose an option (1-9): ");
            try {
                choice = Integer.parseInt(sc.nextLine());
            } catch (NumberFormatException e) {
                choice = -1;
                System.out.println("Invalid input. Please enter a number between 1 and 9.");
            }
            switch (choice) {
                case 1:
                    Student student = new Student();
                    if (registrationManage.newRegistration(student)) {
                        System.out.println("Registration successful!");
                        registrationManage.saveToFile("Registrations.dat");
                    } else {
                        System.out.println("Registration failed.");
                    }
                    break;
                case 2:
                    System.out.print("Enter your Student ID: ");
                    String updateStudentId = sc.nextLine();
                    Student studentToUpdate = registrationManage.getStudentList().findStudentById(updateStudentId);
                    if (studentList.findStudentById(updateStudentId) == null) {
                        System.out.println("This student has not registered yet.");
                    } else {
                        studentToUpdate.updateNewInformationFromKeyboard();
                        registrationManage.updateRegistrationInformation(studentToUpdate);
                        System.out.println("Update successful!");
                    }
                    break;
                case 3:
                    registrationManage.displayRegistrationList();
                    break;
                case 4:
                    if (registrationManage.getStudentList().isEmpty()) {
                        System.out.println("No students have registered yet.");
                        break;
                    } else {
```

```java
                        System.out.print("Enter your Student ID to delete: ");
                        String deleteStudentId = sc.nextLine();
                        Student studentToDelete = registrationManage.getStudentList().findStudentById(deleteStudentId);
                        if (studentToDelete != null) {
                            studentList.displayByStudent(studentToDelete);
                            System.out.print("Are you sure you want to delete this registration? (Y/N): ");
                            String confirm = sc.nextLine();
                            if (confirm.equalsIgnoreCase("Y")) {
                                if (registrationManage.deleteRegistrationInformation(deleteStudentId)) {
                                    System.out.println("The registration has been successfully deleted.");
                                }
                            } else {
                                System.out.println("The registration has not been deleted.");
                            }
                        } else {
                            System.out.println("This student has not registered yet.");
                        }
                    }
                    break;
                case 5:
                    System.out.print("Enter the name to search: ");
                    String searchName = sc.nextLine();
                    StudentList foundStudents = registrationManage.searchParticipantsByName(searchName);
                    if (foundStudents != null) {
                        foundStudents.displayStudents(foundStudents);
                    } else {
                        System.out.println("No one matches the search criteria!");
                    }
                    break;
                case 6:
                    System.out.print("Enter Campus ID to filter (e.g., SE, HE, DE, QE, CE): ");
                    String campusID = sc.nextLine();
                    StudentList filteredList = registrationManage.filterDataByCampus(campusID);
                    if (filteredList != null) {
                        filteredList.displayStudents(filteredList);
                    } else {
                        System.out.println("No one matches the filter criteria!");
                    }
                    break;
                case 7:
                    System.out.print("-----------------------------------------------------------------------\n");
                    System.out.printf("%-10s | %-25s | %-15s\n", "Peak Code", "Number of Participants", "Total Cost");
                    System.out.print("-----------------------------------------------------------------------\n");
                    registrationManage.statisticOfRegistrationByMountainPeak();
```

```java
                System.out.print("----------------------------------------------------------------------\n");
                break;
            case 8:
                int check = registrationManage.saveToFile("RegistrationList.dat");
                if (check == 0) {
                    System.out.println("Registration data has been successfully saved to `registrations.dat`.");
                } else if (check == -1) {
                    System.out.println("Error writing to file.");
                } else if (check == -2) {
                    System.out.println("Error closing file.");
                }
                break;
            case 9:
                if (Validation.isDataChanged()) {
                    System.out.print("Do you want to save the changes before exiting? (Y/N): ");
                    String saveChoice = sc.nextLine();
                    if (saveChoice.equalsIgnoreCase("Y")) {
                        int saveCheck = registrationManage.saveToFile("RegistrationList.txt");
                        if (saveCheck == 0) {
                            System.out.println("Registration data has been successfully saved to `registrations.dat`.");
                        } else if (saveCheck == -1) {
                            System.out.println("Error writing to file.");
                        } else if (saveCheck == -2) {
                            System.out.println("Error closing file.");
                        }
                    }
                }
                System.out.println("Exiting the program. Goodbye!");
                sc.close();
                System.exit(0);
            default:
                System.out.println("This function is not available.");
                break;
        }
    } while (choice != 9);
    }
}
```

Result:

```
Output ×
- D:\ ×    MountainHiking (run) ×
run:
Mountain data loaded successfully, 14 records found.
Registration data loaded successfully, 4 records found.

=========================================================
=       MOUNTAIN HIKING CHALLENGE REGISTRATION      =
=========================================================
| 1. New Registration                                   |
| 2. Update Registration Information                    |
| 3. Display Registered List                            |
| 4. Delete Registration Information                    |
| 5. Search Participants by Name                        |
| 6. Filter Data by Campus                              |
| 7. Statistics of Registration Numbers by Location     |
| 8. Save Data to File                                  |
| 9. Exit the Program                                   |
=========================================================
Please choose an option (1-9): |
```

- **Function 1: New Registration (with Validation)**

  Code:

```java
    public boolean newRegistration(Student student) {
        boolean check = true;
        student.inputStudentInformationFromKeyboard();
        if (studentList.checkExistStudentId(student.getStuID())) {
            System.out.println("This Student ID already exists. Please enter a different one.");
            check = false;
        } else {
            student.setTuitionFee(studentList.calculateTuitionFee(student.getPhoneNum()));
            studentList.add(student);
        }
        Validation.dataChanged = true;
        return check;
    }
```

```java
public class Validation {
    public static final String STUDENT_ID_PATTERN = "^(SE|HE|DE|QE|CE)\\d{6}$";
    public static final String NAME_PATTERN = "^.{2,20}$";
    public static final String PHONE_NUMBER_PATTERN = "^(09|08|03|07|05)\\d{8}$";
    // Viettel: 09[6-8], 086, 03[2-9]
    // VNPT (VinaPhone): 09[14], 088, 08[1-5]
    public static final String DISCOUNT_PHONE_PATTERN = "^((09[6-8]|086|03[2-9])|(09[14]|088|08[1-5]))\\d{7}$";
    public static final String EMAIL_PATTERN = "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}$";
    private static MountainList mountainList = new MountainList();
    public static boolean dataChanged;

    public static void setMountainList(MountainList mList) {
        mountainList = mList;
    }

    public static boolean isValidStudentId(String studentId) {
        return studentId != null && studentId.matches(STUDENT_ID_PATTERN);
    }

    public static boolean isValidName(String name) {
        return name != null && name.matches(NAME_PATTERN);
    }

    public static boolean isValidPhoneNumber(String phoneNumber) {
        return phoneNumber != null && phoneNumber.matches(PHONE_NUMBER_PATTERN);
    }

    public static boolean isDiscountablePhoneNumber(String phoneNumber) {
        return phoneNumber != null && phoneNumber.matches(DISCOUNT_PHONE_PATTERN);
    }

    public static boolean isValidEmail(String email) {
        return email != null && email.matches(EMAIL_PATTERN);
    }

    public static boolean isValidMountainCode(String inputCode) {
        boolean check = false;
        if (inputCode == null || inputCode.trim().isEmpty()) {
            check = false;
        }
        if (mountainList == null || mountainList.isEmpty()) {
            check = false;
        }
        for (Mountain m : mountainList) {
            if (m.getMountainCode().equalsIgnoreCase(inputCode)) {
                check = true;
                break;
            }
        }
        return check;
    }
```

Result:

```
Output
- D:\ MountainHiking (run)
run:
Mountain data loaded successfully, 14 records found.
Registration data loaded successfully, 4 records found.

=========================================================
=       MOUNTAIN HIKING CHALLENGE REGISTRATION      =
=========================================================
| 1. New Registration                                   |
| 2. Update Registration Information                    |
| 3. Display Registered List                            |
| 4. Delete Registration Information                    |
| 5. Search Participants by Name                        |
| 6. Filter Data by Campus                              |
| 7. Statistics of Registration Numbers by Location     |
| 8. Save Data to File                                  |
| 9. Exit the Program                                   |
=========================================================
Please choose an option (1-9): 1
Enter your Student ID: SE123456
Enter your Name: MinhNH
Enter your Phone Number: 0911850906
Enter your Email: hming150906@gmail.com
Enter Mountain Peak Code (e.g., 01): 01
Invalid Mountain Peak Code. Please try again.
Enter Mountain Peak Code (e.g., 01): 1
Registration successful!

=========================================================
=       MOUNTAIN HIKING CHALLENGE REGISTRATION      =
=========================================================
| 1. New Registration                                   |
| 2. Update Registration Information                    |
| 3. Display Registered List                            |
| 4. Delete Registration Information                    |
```

- **Function 2: Update Registration**

Code:

```java
public boolean updateRegistrationInformation(Student student) {
    boolean check = true;
    for (Student s : this.studentList) {
        if (s.getStuID().equals(student.getStuID())) {
            if (student.getName() != null) s.setName(student.getName());
            if (student.getPhoneNum() != null) s.setPhoneNum(student.getPhoneNum());
            if (student.getEmail() != null) s.setEmail(student.getEmail());
            if (student.getMountainCode() != null) s.setMountainCode(student.getMountainCode());
            s.setTuitionFee(student.getTuitionFee());
            break;
        }
    }
    Validation.dataChanged = true;
    return check;
}
```

```java
public static boolean isDataChanged() {
    return dataChanged;
}
```

Result:

```
Output
- D:\ ×    MountainHiking (run) ×
This function is not available.

=========================================================
=       MOUNTAIN HIKING CHALLENGE REGISTRATION      =
=========================================================
| 1. New Registration                                  |
| 2. Update Registration Information                   |
| 3. Display Registered List                           |
| 4. Delete Registration Information                   |
| 5. Search Participants by Name                       |
| 6. Filter Data by Campus                             |
| 7. Statistics of Registration Numbers by Location    |
| 8. Save Data to File                                 |
| 9. Exit the Program                                  |
=========================================================
Please choose an option (1-9): 2
Enter your Student ID: SE203563
Enter your new Name: MinhNH
Enter your new Phone Number:
Enter your new Email:
Enter new Mountain Peak Code (e.g., 01): 12
Update successful!

=========================================================
=       MOUNTAIN HIKING CHALLENGE REGISTRATION      =
=========================================================
| 1. New Registration                                  |
| 2. Update Registration Information                   |
| 3. Display Registered List                           |
| 4. Delete Registration Information                   |
| 5. Search Participants by Name                       |
| 6. Filter Data by Campus                             |
| 7. Statistics of Registration Numbers by Location    |
| 8. Save Data to File                                 |
```

- **Function 3: Display registration list**

Code:

```java
    public void displayRegistrationList() {
        if (studentList.isEmpty()) {
            System.out.println("No students have registered yet.");
        } else {
            System.out.print("-----------------------------------------------------------------------------------------\n");
            System.out.printf("%-15s | %-25s | %-15s | %-10s | %-10s\n", "Student ID", "Name", "Phone", "Peak Code", "Fee");
            System.out.print("-----------------------------------------------------------------------------------------\n");
            for (Student s : studentList) {
                System.out.printf(s.toString() + "\n");
            }
            System.out.print("-----------------------------------------------------------------------------------------\n");
        }
    }
```

```java
    @Override
    public String toString() {
        return String.format("%-15s | %-25s | %-15s | %-10s | %-10s", studentID, name, phoneNum, "MT" + mountainCode, tuitionFee);
    }
}
```

Result:

```
Output ×
- D:\ ×    MountainHiking (run) ×
Enter your new Phone Number:
Enter your new Email:
Enter new Mountain Peak Code (e.g., 01): 12
Update successful!

=========================================================
=       MOUNTAIN HIKING CHALLENGE REGISTRATION      =
=========================================================
| 1. New Registration                                   |
| 2. Update Registration Information                    |
| 3. Display Registered List                            |
| 4. Delete Registration Information                    |
| 5. Search Participants by Name                        |
| 6. Filter Data by Campus                              |
| 7. Statistics of Registration Numbers by Location     |
| 8. Save Data to File                                  |
| 9. Exit the Program                                   |
=========================================================
Please choose an option (1-9): 3
-----------------------------------------------------------------------------------------
Student ID      | Name                      | Phone           | Peak Code   | Fee
-----------------------------------------------------------------------------------------
SE203563        | MinhNH                    | 0911850906      | MT12        | 3900000
SE203676        | Dung                      | 0311850906      | MT3         | 6000000
HE123456        | Than Thi Ngoc             | 0765432123      | MT10        | 6000000
QE203563        | Le Thi Minh               | 0567890113      | MT7         | 6000000
SE123456        | MinhNH                    | 0911850906      | MT1         | 3900000
-----------------------------------------------------------------------------------------
```

- **Function 4: Delete registration**

Code:

```java
public boolean deleteRegistrationInformation(String studentId) {
    boolean check = false;
    Student studentToDelete = studentList.findStudentById(studentId);
    if (studentToDelete != null) {
        studentList.displayByStudent(studentToDelete);
        studentList.remove(studentToDelete);
        check = true;
    }
    Validation.dataChanged = true;
    return check;
}
```

```java
public Student findStudentById(String studentId) {
    Student foundStudent = null;
    for (Student s : this) {
        if (s.getStuID().equals(studentId)) {
            foundStudent = s;
            break;
        }
    }
    return foundStudent;
}
```

```java
public void displayByStudent(Student s) {
    System.out.println("----------------------------------------------------------------");
    System.out.printf("Student ID: %s\n", s.getStuID());
    System.out.printf("Name      : %s\n", s.getName());
    System.out.printf("Phone     : %s\n", s.getPhoneNum());
    System.out.printf("Moutain   : MT%s\n", s.getMountainCode());
    System.out.printf("Fee       : %s\n", s.getTuitionFee());
    System.out.println("----------------------------------------------------------------");
}
```

Result:

```
Output ×
- D:\ ×   MountainHiking (run) ×

=====================================================
=       MOUNTAIN HIKING CHALLENGE REGISTRATION      =
=====================================================
| 1. New Registration                               |
| 2. Update Registration Information                |
| 3. Display Registered List                        |
| 4. Delete Registration Information                |
| 5. Search Participants by Name                    |
| 6. Filter Data by Campus                          |
| 7. Statistics of Registration Numbers by Location |
| 8. Save Data to File                              |
| 9. Exit the Program                               |
=====================================================
Please choose an option (1-9): 4
Enter your Student ID to delete: SE123456
----------------------------------------------------------------
Student ID: SE123456
Name      : MinhNH
Phone     : 0911850906
Moutain   : MT1
Fee       : 3900000
----------------------------------------------------------------
Are you sure you want to delete this registration? (Y/N): Y
----------------------------------------------------------------
Student ID: SE123456
Name      : MinhNH
Phone     : 0911850906
Moutain   : MT1
Fee       : 3900000
----------------------------------------------------------------
The registration has been successfully deleted.

=====================================================
=       MOUNTAIN HIKING CHALLENGE REGISTRATION      =
                                                MountainHiki
```

- **Function 5: Search participants by name**

Code:

```java
    public StudentList searchParticipantsByName(String name) {
        StudentList searchResults = new StudentList();
        for (Student s : this.studentList) {
            if (s.getName().toLowerCase().contains(name.toLowerCase())) {
                searchResults.add(s);
            }
        }
        return searchResults;
    }
```

```java
    public void displayStudents(StudentList specialList) {
        System.out.print("--------------------------------------------------------------------------------\n");
        System.out.printf("%-15s | %-30s | %-15s | %-10s | %-10s\n", "Student ID", "Name", "Phone", "Peek Code", "Fee");
        System.out.print("--------------------------------------------------------------------------------\n");
        for (Student s : specialList) {
            System.out.printf(s.toString() + "\n");
        }
        System.out.print("--------------------------------------------------------------------------------\n");
    }
}
```

Result:

```
Output ×
- D:\ ×   MountainHiking (run) ×
Name        : MinhNH
Phone       : 0911850906
Moutain     : MT1
Fee         : 3900000
-------------------------------------------------------------------
The registration has been successfully deleted.

=========================================================
=        MOUNTAIN HIKING CHALLENGE REGISTRATION        =
=========================================================
| 1. New Registration                                   |
| 2. Update Registration Information                    |
| 3. Display Registered List                            |
| 4. Delete Registration Information                    |
| 5. Search Participants by Name                        |
| 6. Filter Data by Campus                              |
| 7. Statistics of Registration Numbers by Location     |
| 8. Save Data to File                                  |
| 9. Exit the Program                                   |
=========================================================
Please choose an option (1-9): 5
Enter the name to search: Minh
--------------------------------------------------------------------------------
Student ID      | Name                           | Phone           | Peek Code   | Fee
--------------------------------------------------------------------------------
SE203563        | MinhNH                         | 0911850906      | MT12        | 3900000
QE203563        | Le Thi Minh                    | 0567890113      | MT7         | 6000000
--------------------------------------------------------------------------------
```

- **Function 6: Filter data by campus**

  Code:

```java
    public StudentList filterDataByCampus(String campusID) {
        StudentList studentByCampusList = new StudentList();
        String campusCode = campusID.substring(0, 1).toUpperCase();
        for (Student s : this.studentList) {
            if (s.getStuID().toUpperCase().startsWith(campusCode)) {
                studentByCampusList.add(s);
            }
        }
        return studentByCampusList;
    }
```

```java
    public void displayStudents(StudentList specialList) {
        System.out.print("--------------------------------------------------------------------------------\n");
        System.out.printf("%-15s | %-25s | %-15s | %-10s | %-10s\n", "Student ID", "Name", "Phone", "Peek Code", "Fee");
        System.out.print("--------------------------------------------------------------------------------\n");
        for (Student s : specialList) {
            System.out.printf(s.toString() + "\n");
        }
        System.out.print("--------------------------------------------------------------------------------\n");
    }
}
```

  Result:

```
Output
- D:\ ×   MountainHiking (run) ×
run:
Mountain data loaded successfully, 14 records found.
Registration data loaded successfully, 5 records found.

=========================================================
=       MOUNTAIN HIKING CHALLENGE REGISTRATION      =
=========================================================
| 1. New Registration                                   |
| 2. Update Registration Information                    |
| 3. Display Registered List                            |
| 4. Delete Registration Information                    |
| 5. Search Participants by Name                        |
| 6. Filter Data by Campus                              |
| 7. Statistics of Registration Numbers by Location     |
| 8. Save Data to File                                  |
| 9. Exit the Program                                   |
=========================================================
Please choose an option (1-9): 6
Enter Campus ID to filter (e.g., SE, HE, DE, QE, CE): SE
--------------------------------------------------------------------------------
Student ID      | Name                      | Phone           | Peek Code   | Fee
--------------------------------------------------------------------------------
SE203563        | Minh Nguyen Hoang         | 0911850906      | MT2         | 3900000
SE203676        | Dung                      | 0311850906      | MT3         | 6000000
SE123456        | MinhNH                    | 0911850906      | MT1         | 3900000
--------------------------------------------------------------------------------
```

- **Function 7: Statistics of registration number by location**

Code:

```java
    public void statisticOfRegistrationByMountainPeak() {
        int count = 0;
        for (Mountain m : mountainList) {
            int participantCount = 0;
            int totalCost = 0;
            for (Student s : studentList) {
                if (s.getMountainCode().equals(m.getMountainCode())) {
                    participantCount++;
                    totalCost += s.getTuitionFee();
                }
            }
            if (participantCount > 0) {
                mountainList.displayStatisticOfRegistrationByMountainPeak(m.getMountainCode(), participantCount, totalCost);
                count++;
            }
        }
        if (count == 0) {
            System.out.println("No registrations found for any mountain peak.");
        }
    }
```

```java
    public void displayStatisticOfRegistrationByMountainPeak(String peakCode, int numberOfParticipants, int totalCost) {
        System.out.printf("%-10s | %-25s | %-15s", "MT" + peakCode, numberOfParticipants, totalCost);
        System.out.println();
    }
}
```

Result:

```
Output ×
- D:\ ×    MountainHiking (run) ×
--------------------------------------------------------------------------------

=========================================================
=        MOUNTAIN HIKING CHALLENGE REGISTRATION       =
=========================================================
| 1. New Registration                                   |
| 2. Update Registration Information                    |
| 3. Display Registered List                            |
| 4. Delete Registration Information                    |
| 5. Search Participants by Name                        |
| 6. Filter Data by Campus                              |
| 7. Statistics of Registration Numbers by Location     |
| 8. Save Data to File                                  |
| 9. Exit the Program                                   |
=========================================================
Please choose an option (1-9): 7
----------------------------------------------------------------------
Peak Code  | Number of Participants    | Total Cost
----------------------------------------------------------------------
MT1        | 1                         | 3900000
MT2        | 1                         | 3900000
MT3        | 1                         | 6000000
MT7        | 1                         | 6000000
MT10       | 1                         | 6000000
----------------------------------------------------------------------
```

- **Function 8: Save to file "registration.dat"**

  Code:

```java
    public int saveToFile(String path) {
        int check = 0;
        PrintWriter f = null;
        try {
            f = new PrintWriter(path);
            for (Student u : this.studentList) {
                String s = u.getStuID() + "," + u.getName() + "," + u.getPhoneNum() + "," + u.getEmail() + "," + u.getMountainCode() + "," + u.getTuitionFee() + "\n";
                f.print(s);
                f.flush();
            }
        } catch (Exception e) {
            check = -1;
            //
        } finally {
            try {
                if(f!=null) f.close();
            } catch (Exception e) {
                check = -2;
            }
        }
        return check;
    }
}
```

  Result:

```
Output ×
- D:\ ×    MountainHiking (run) ×
This student has not registered yet.

=========================================================
=        MOUNTAIN HIKING CHALLENGE REGISTRATION       =
=========================================================
| 1. New Registration                                   |
| 2. Update Registration Information                    |
| 3. Display Registered List                            |
| 4. Delete Registration Information                    |
| 5. Search Participants by Name                        |
| 6. Filter Data by Campus                              |
| 7. Statistics of Registration Numbers by Location     |
| 8. Save Data to File                                  |
| 9. Exit the Program                                   |
=========================================================
Please choose an option (1-9): 8
Registration data has been successfully saved to `registrations.dat`.
```

- **Function 9: Save Prompt on Exit**

  Code:

```java
    public int saveToFile(String path) {
        int check = 0;
        PrintWriter f = null;
        try {
            f = new PrintWriter(path);
            for (Student u : this.studentList) {
                String s = u.getStuID() + "," + u.getName() + "," + u.getPhoneNum() + "," + u.getEmail() + "," + u.getMountainCode() + "," + u.getTuitionFee() + "\n";
                f.print(s);
                f.flush();
            }
        } catch (Exception e) {
            check = -1;
            //
        } finally {
            try {
                if(f!=null) f.close();
            } catch (Exception e) {
                check = -2;
            }
        }
        return check;
    }
}
```

  Result:

```
Output ×
- D:\ ×    MountainHiking (run) ×
| 9. Exit the Program                                  |
=======================================================
Please choose an option (1-9): 2
Enter your Student ID: SE203563
Enter your new Name: MinhNH
Enter your new Phone Number: 0911850906
Enter your new Email:
Enter new Mountain Peak Code (e.g., 01):
Update successful!

=======================================================
=       MOUNTAIN HIKING CHALLENGE REGISTRATION      =
=======================================================
| 1. New Registration                                 |
| 2. Update Registration Information                  |
| 3. Display Registered List                          |
| 4. Delete Registration Information                  |
| 5. Search Participants by Name                      |
| 6. Filter Data by Campus                            |
| 7. Statistics of Registration Numbers by Location   |
| 8. Save Data to File                                |
| 9. Exit the Program                                 |
=======================================================
Please choose an option (1-9): 9
Do you want to save the changes before exiting? (Y/N): Y
Registration data has been successfully saved to `registrations.dat`.
Exiting the program. Goodbye!
BUILD SUCCESSFUL (total time: 31 seconds)
```

6. Self-Evaluation Checklist

This table presents a self-assessment based on the 20 provided grading criteria.

Table 1. Self - Evaluation

Number of Criterion	Criterion	Self-Assessment	Notes & Implementation Details
1	Algorithm design & main menu	Achieved	The SystemUsing.java class contains the main method with a do-while loop and a switch-case for the 1-9 menu options. It handles NumberFormatException for invalid input and repeats the menu.
2	Use Case diagram	Achieved	A Use Case diagram was designed featuring one actor (Operator) and nine use cases, including an <<include>> relationship to "Validate Input".
3	Class diagram	Achieved	A Class diagram was designed, including Student, Mountain, StudentList, MountainList, RegistrationManage, and Validation classes. "Has-a" relationships are clearly defined.
4	Sequence diagram	Not Implemented	This criterion was optional and therefore not implemented in the project.
5	Well-defined classes	Achieved	The Student and Mountain model classes are clearly defined with necessary attributes. StudentList and MountainList act as collection classes to manage them.
6	Encapsulation	Achieved	All fields in the model classes (Student, Mountain) are declared private and are accessed exclusively through public getter and setter methods.

7	Constructors / toString()	Achieved	Model classes include both default (no-argument) and parameterized constructors. The toString() method is overridden in Student to format output for table display.
8	Inheritance	Achieved	Inheritance is used where StudentList extends ArrayList<Student> and MountainList extends ArrayList<Mountain> to leverage existing list functionalities.
9	Display data	Achieved	Data is displayed in well-aligned, formatted tables using System.out.printf with fixed column widths for clarity.
10	Add object (Student)	Achieved	The add function validates all inputs using REGEX, checks for duplicate Student IDs, and verifies the Mountain Code's existence before creating a new Student object.
11	Find & Update (Student)	Achieved	The update function allows finding a student by ID. During the update process, the user can press Enter to leave a field blank, which keeps the original data intact.
12	Delete object by ID	Achieved	The delete function requires a Student ID, displays the information of the student to be deleted, and requires a (Y/N) confirmation before proceeding.
13	Filter by criteria (Campus)	Achieved	A function to filter the student list by campus (based on the first two characters of the Student ID) and display the results has been implemented.
14	Load Data (Mountain)	Achieved	The application reads and loads mountain data from MountainList.csv

			upon startup, making the list of valid mountains available for validation.
15	Search object (Student)	Achieved	The search functionality supports finding a student by an exact ID match and by a partial, case-insensitive name match (contains).
16	Statistics	Achieved	The statistics function correctly calculates the number of participants and the total tuition fee for each mountain peak and displays a summary table.
17	Save data to file	Achieved	The application provides a function to save the current list of student registrations to a file named RegistrationList.dat.
18	Exit Prompt	Achieved	On exit, the program checks a dataChanged flag. If changes have been made, it prompts the user to save before quitting, preventing data loss.
19	Naming conventions	Achieved	The code adheres to Java naming standards: PascalCase for class names and camelCase for methods and variables. The package structure is logical.
20	Clean & clear code, report	Achieved	The code is well-formatted with comments for complex logic. The report is written concisely and clearly, covering all required sections.

--- END ---